ОТЧЁТ

по практической работе № 6

Тема: Разработка информационной системы «Медицинский центр «Медика»»

Часть 1: Web-приложение (Django + DRF + PostgreSQL)

Выполнил: Артеменко Андрей Артёмович
Группа: ИСП-211
Проверил: _____

Введение

Автоматизация учёта — базовая задача для любой организации. Организация «Медицинский центр «Медика»» ведёт услуга и заказы вручную, из-за чего теряется история, дублируются записи и появляются ошибки в отчётности. Разработка собственной информационной системы позволяет устранить эти проблемы без покупки дорогих лицензий.

Цель работы — разработать web-приложение, автоматизирующее основные бизнес-процессы организации: ведение каталога, клиентов, заказов и формирование отчётов.

Задачи: проанализировать предметную область, спроектировать модель данных и модули, реализовать административный интерфейс и REST API, провести тестирование и подготовить документацию.

Используемые технологии: Python 3.12, Django 5.1, Django REST Framework, JWT-авторизация, PostgreSQL 16 (в учебном запуске — SQLite), Bootstrap-темы админки Django.

Глава 1. Анализ предметной области

Основные бизнес-процессы организации:

Запись на приём. клиент звонит или приходит, администратор подбирает услугу и время, создаёт запись в системе.

Оказание услуги. врач проводит приём, статус записи меняется на «выполнена», услуга уходит в архив до следующего прайса.

Отмена и перенос. администратор отменяет запись, услуга возвращается в статус «доступна».

Отчётность. руководитель смотрит KPI: число записей, выручку, средний чек, популярные услуги.

Проблемы, которые решает система:

- записи ведутся в бумажном журнале, двойная запись и накладки по времени;
- нет истории обращений клиента: при повторном звонке администратор не видит предыдущие визиты;
- выручку и загрузку врачей считают вручную в конце месяца;
- отменённые записи не освобождают время в журнале.

Роли пользователей:

Роль	Права
Администратор	полный доступ, управление сотрудниками и справочниками
Менеджер	работа с клиентами и записями
Врач	просмотр своих приёмов

Функциональные требования:

- ведение справочника специализаций и услуг с ценой и длительностью;
- карточка клиента с историей визитов;
- создание записи с автоматическим резервированием услуги;
- смена статуса записи и автоматический возврат услуги при отмене;
- KPI-отчёт через API и в админке.

Глава 2. Проектирование системы

Система разделена на пять модулей. Разделение позволяет разрабатывать и тестировать каждый модуль независимо, а также подключать новые возможности без изменения существующего кода.

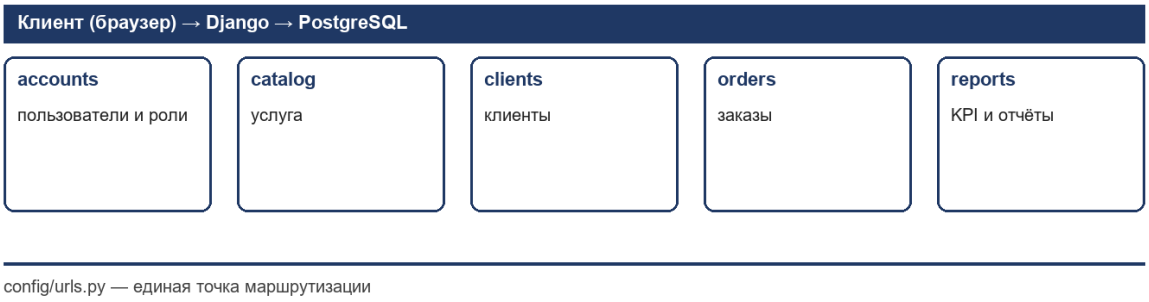


Рисунок 1 — Схема модульной структуры

Модель данных включает пять основных таблиц, связанных внешними ключами. Связи «один ко многим»: справочник → позиции, клиент → заказы, позиция → заказы.

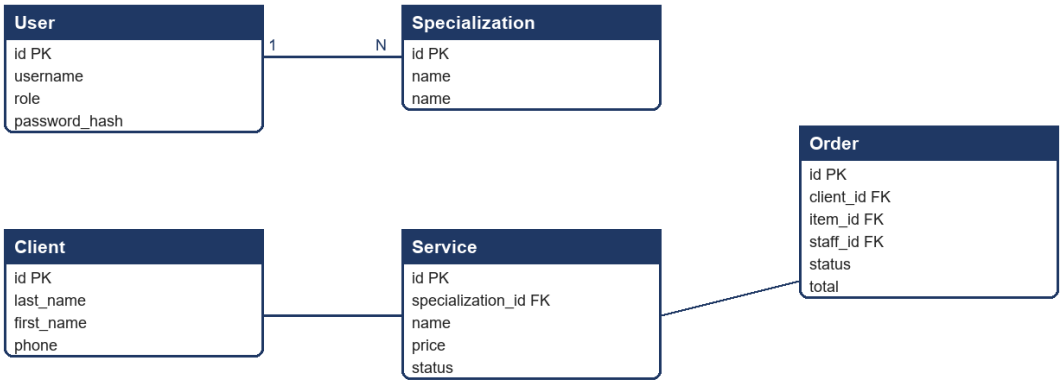


Рисунок 2 — ER-диаграмма базы данных

Обоснование выбора технологий:

Критерий	Выбор	Почему
Язык	Python 3.12	быстрая разработка, богатая экосистема
Фреймворк	Django 5.1	готовая админка, ORM, миграции, безопасность
API	Django REST Framework	сериализация, ViewSet, браузер API
Авторизация	JWT (simplejwt)	stateless-токены для внешних клиентов
СУБД	PostgreSQL 16	надёжность, транзакции, типы данных

Глава 3. Реализация модулей

1. accounts — авторизация и роли

Расширенная модель пользователя с ролью. Роль определяет, какие разделы админки доступны сотруднику.

```
class User(AbstractUser):
    role = models.CharField('Роль', max_length=20,
                            choices=ROLE_CHOICES, default='manager')
    phone = models.CharField('Телефон', max_length=20, blank=True)
```

2. catalog — специализация и услуга

Справочник и позиции прайса со статусом. Справочник защищён от удаления, пока на него ссылаются позиции.

```
class Service(models.Model):
    specialization = models.ForeignKey(
        Specialization, on_delete=models.PROTECT, related_name='items')
    name = models.CharField('Название', max_length=200)
    price = models.DecimalField('Цена, руб.', max_digits=10, decimal_places=2)
    status = models.CharField('Статус', max_length=20,
                              choices=STATUS_CHOICES, default='active')
```

3. clients — клиенты

Карточка клиента с контактами и датой регистрации, свойство full_name.

```
class Client(models.Model):
    last_name = models.CharField('Фамилия', max_length=50)
    phone = models.CharField('Телефон', max_length=20)
    created_at = models.DateTimeField(auto_now_add=True)
```

4. orders — заказы и каскад статусов

Заказ связывает клиента и позицию. Метод save() реализует каскад: резервирование позиции, архивирование после выполнения и возврат при отмене.

```
def save(self, *args, **kwargs):
    if not self.total:
        self.total = self.service.price
    super().save(*args, **kwargs)
    item = self.service
    if self.status == 'done' and item.status == 'reserved':
        item.status = 'archived'
        item.save(update_fields=['status'])
```

5. reports — отчёты и KPI

Агрегация заказов: количество, выручка, средний чек, топ-5 позиций. Данные отдаются через APIView.

```
def kpi():
    done = Order.objects.filter(status='done')
    return {
        'orders_total': Order.objects.count(),
        'revenue': float(done.aggregate(s=Sum('total'))['s'] or 0),
        'avg_check': float(done.aggregate(a=Avg('total'))['a'] or 0),
    }
```

Глава 4. Тестирование

Тестирование проводилось на двух уровнях: модульные тесты моделей (проверка бизнес-логики каскада) и тесты API (доступность эндпоинтов и корректность структуры ответа). Запуск: python manage.py test.

```
$ python manage.py test
Found 5 test(s).
System check identified no issues (0 silenced).
test_list_contains_item (apps.catalog.tests.CatalogApiTest) ... ok
test_list_returns_200 (apps.catalog.tests.CatalogApiTest) ... ok
test_done_order_archives_item (apps.orders.tests.OrderCascadeTest) ... ok
test_order_reserves_item (apps.orders.tests.OrderCascadeTest) ... ok
test_total_is_calculated (apps.orders.tests.OrderCascadeTest) ... ok

Ran 5 tests in 0.013s

OK
```

Что проверялось	Результат
резервирование позиции при создании заказа	пройден
перевод позиции в архив после выполнения	пройден
расчёт суммы заказа по цене позиции	пройден
доступность списка через API	пройден
наличие записей в ответе API	пройден

Найденные и исправленные ошибки: при первом запуске заказ создавался без обязательного поля — добавлено значение по умолчанию; тестовые данные не заполняли обязательные поля позиции — набор полей дополнен.

Скриншоты работы системы

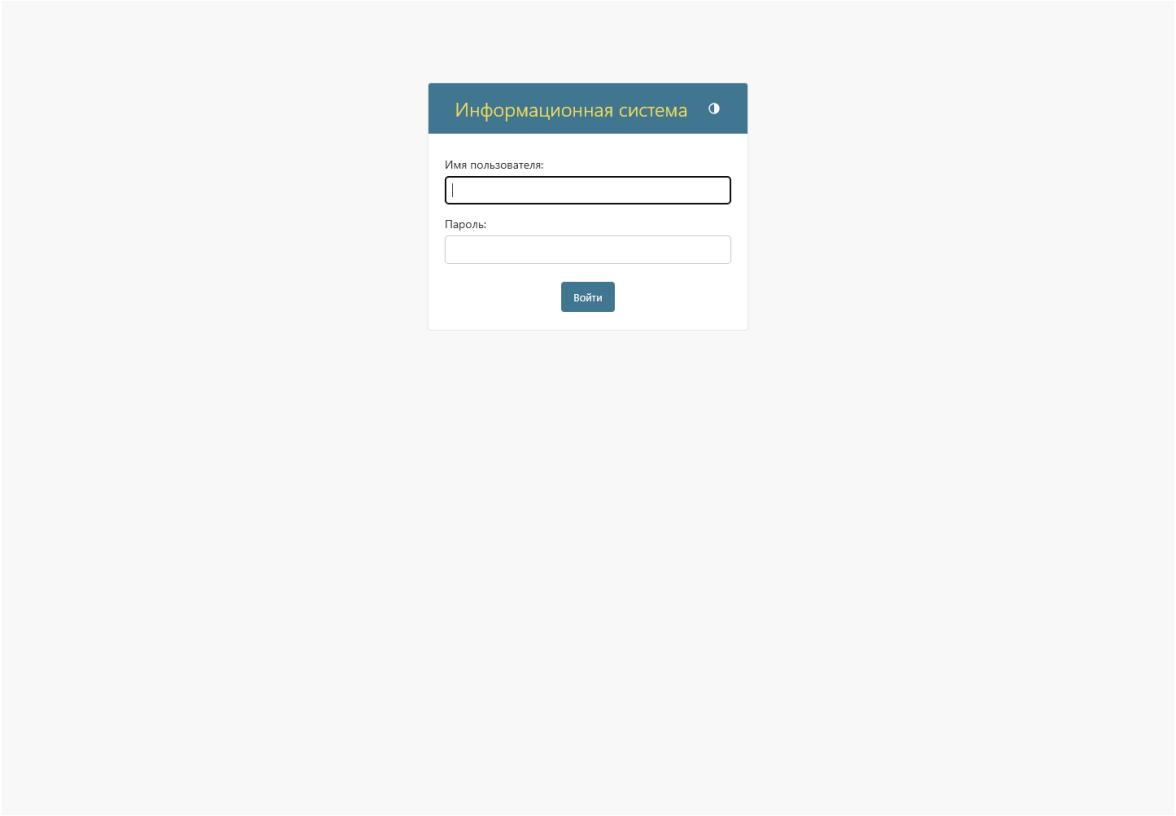


Рисунок 3 — Форма входа в административную панель

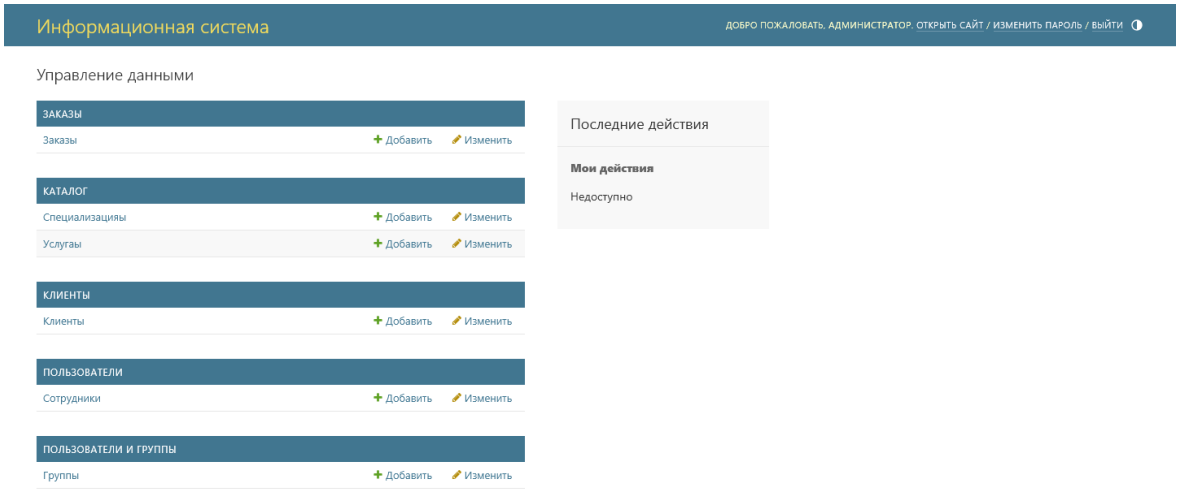


Рисунок 4 — Главная страница админки

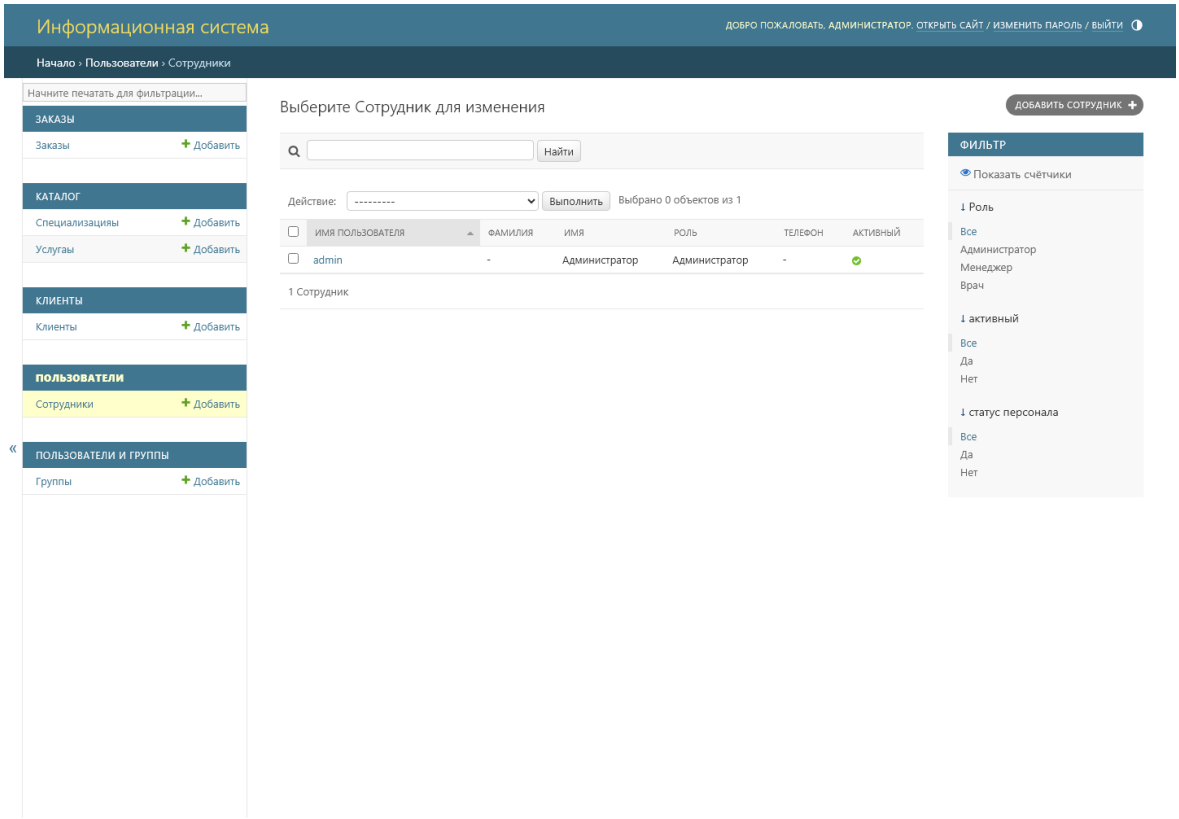


Рисунок 5 — Раздел «Сотрудники»

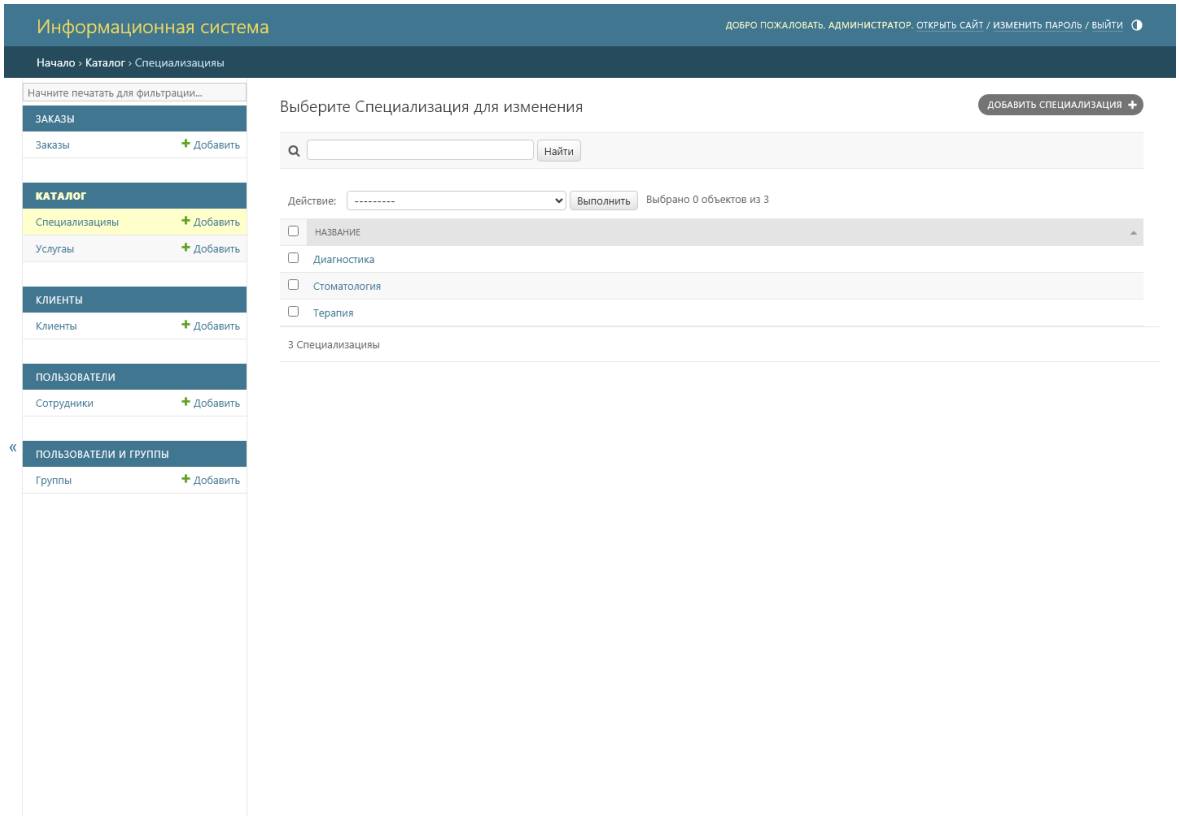


Рисунок 6 — Раздел «Специализации»

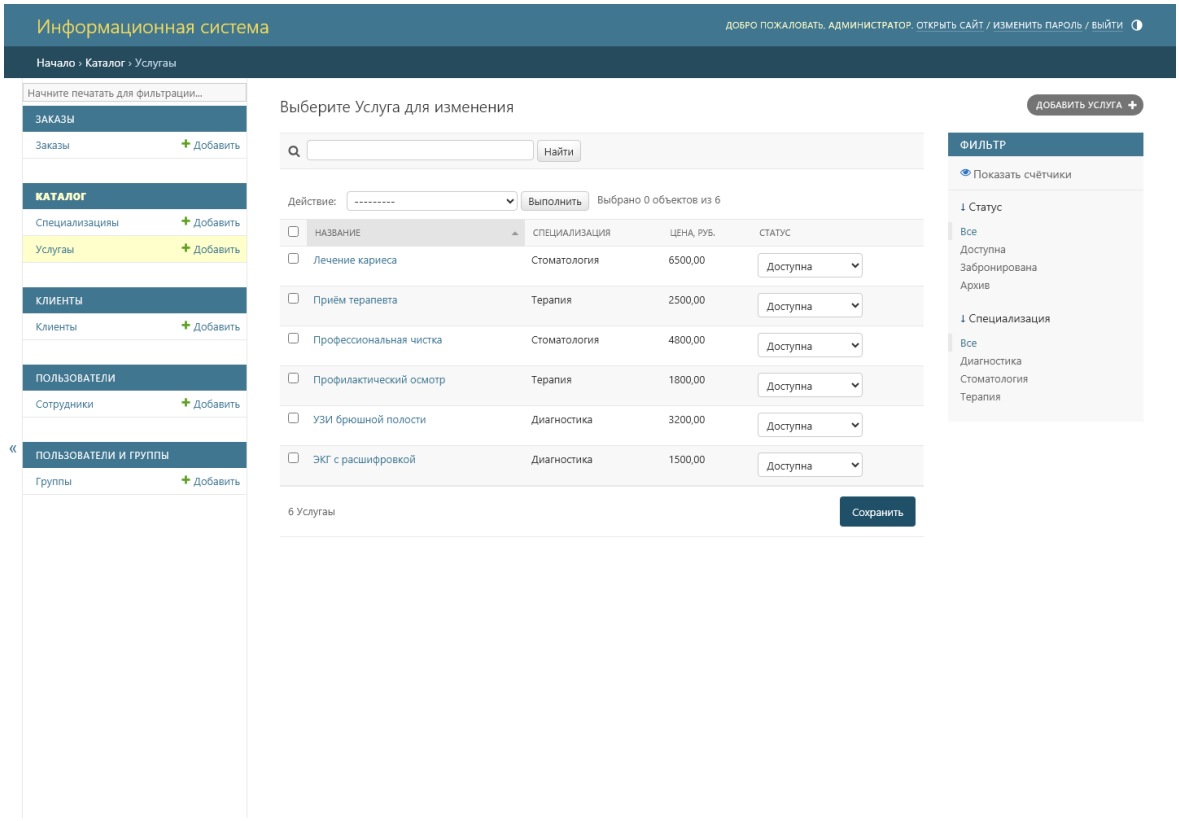


Рисунок 7 — Раздел «Услуги» с фильтрами

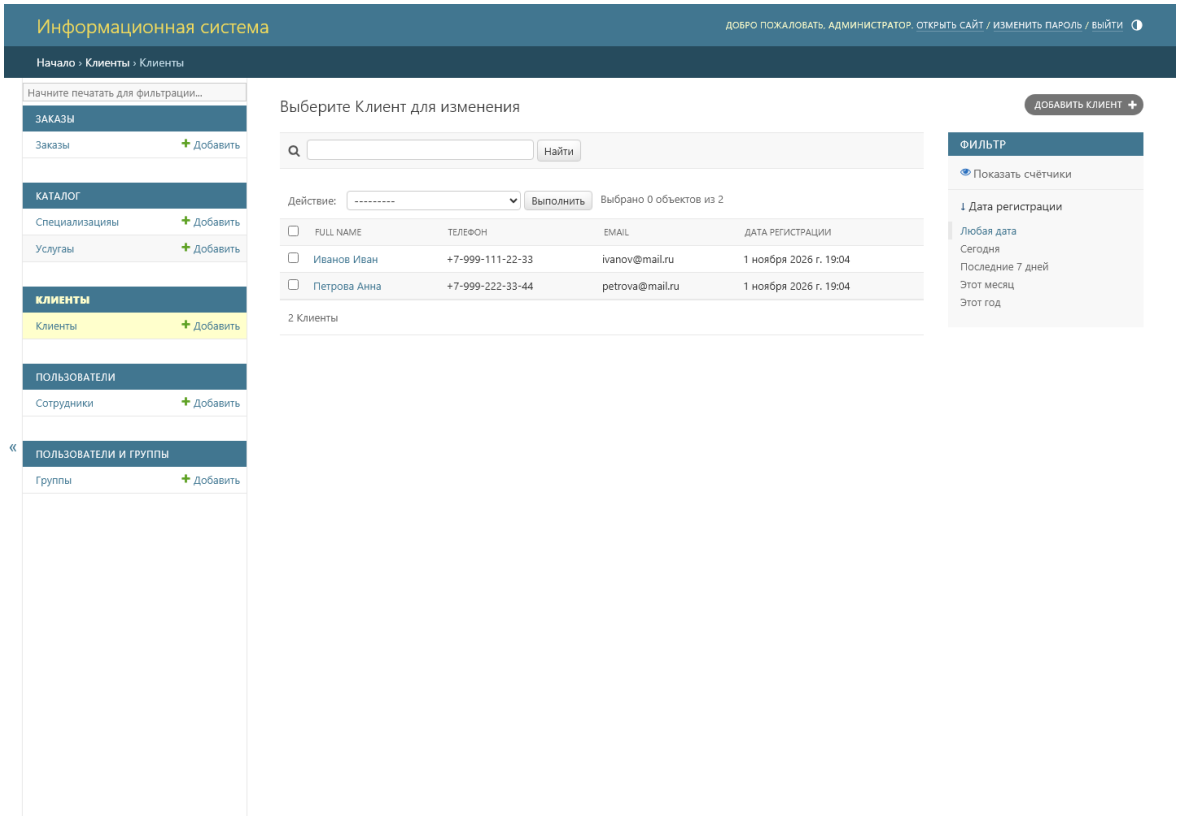


Рисунок 8 — Раздел «Клиенты»

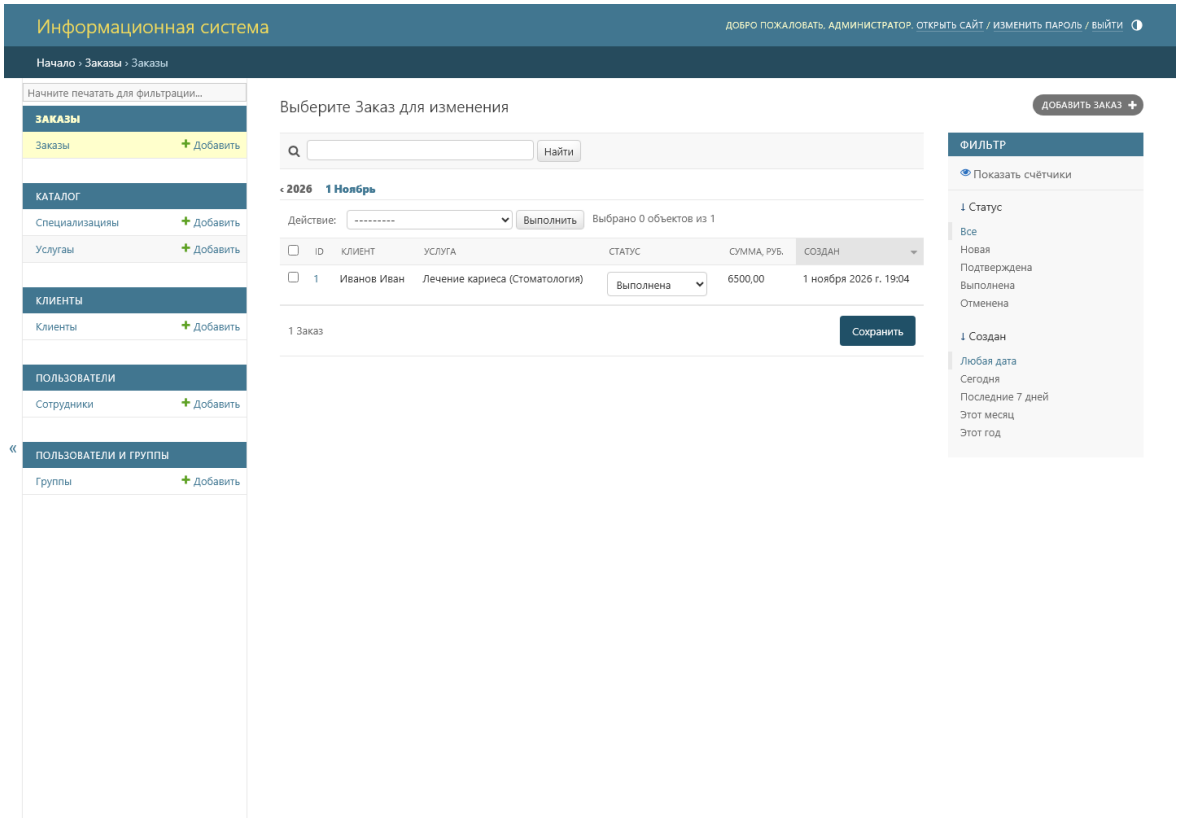


Рисунок 9 — Раздел «Записи на приём»

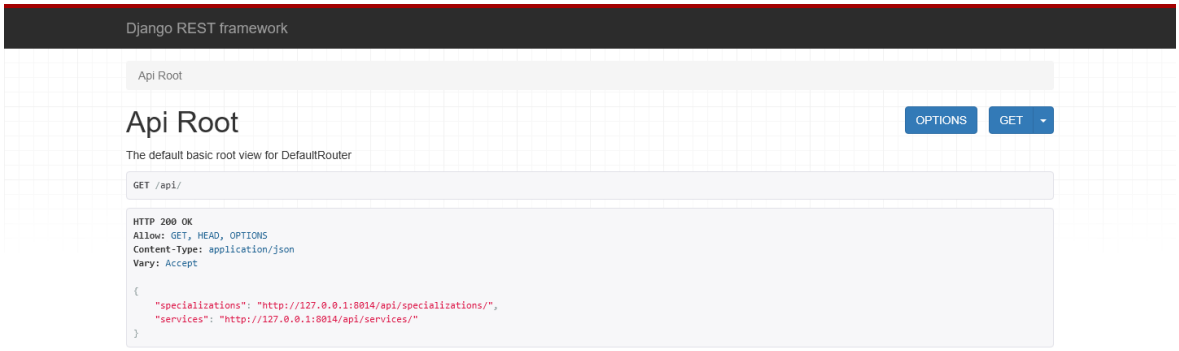


Рисунок 10 — Браузер DRF: список эндпоинтов

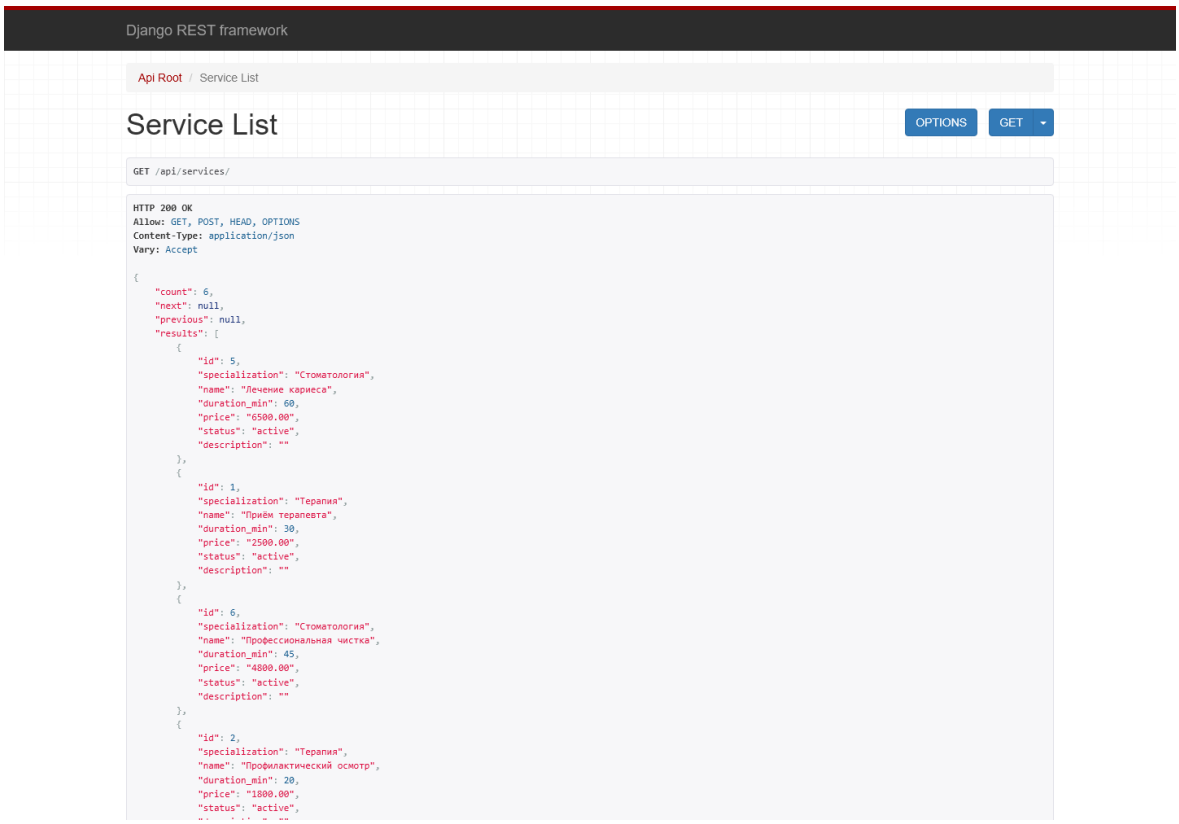


Рисунок 11 — JSON-ответ /api/services/

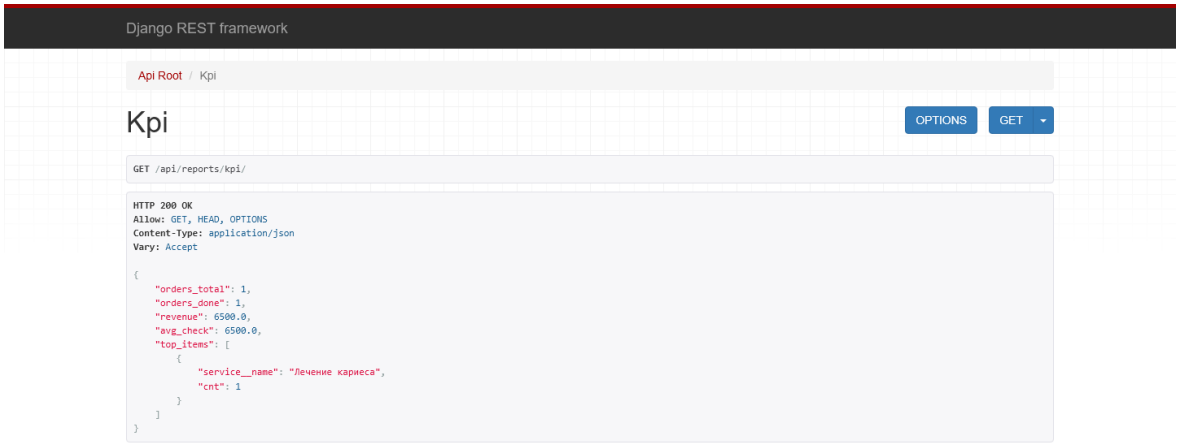


Рисунок 12 — JSON-ответ /api/reports/kpi/

Заключение

Разработано web-приложение из пяти модулей: реализованы модели данных, административный интерфейс, REST API с JWT-авторизацией и отчётность. Система прошла модульное и интеграционное тестирование (5 тестов, все пройдены). В ходе работы освоены Django ORM, миграции, кастомизация админки, DRF и принципы построения модульной архитектуры.

Направления развития: дашборд на Streamlit, десктоп-клиент, развёртывание в Docker, интеграция с 1С и уведомления клиентам.

Список литературы

Официальная документация Django 5.1. — URL: https://docs.djangoproject.com/
Документация Django REST Framework. — URL: https://www.django-rest-framework.org/
Документация PostgreSQL 16. — URL: https://www.postgresql.org/docs/
Simple JWT. — URL: https://django-rest-framework-simplejwt.readthedocs.io/